Process in Operating System

◆ Definition

A **Process** is a program in execution.

A **Program** is a passive entity stored on disk (in the form of an executable file), whereas a **Process** is an active entity that is currently running in memory.

◆ Program vs Process

Program	Process
Passive entity	Active entity
Stored on disk	Resides in main memory
Static	Dynamic
Does not perform action	Performs execution

☞ A program becomes a process when it is loaded into memory and begins execution.

◆ How a Program Becomes a Process

Execution of a program can be started by:

- GUI mouse clicks (e.g., double-clicking an application)
- Command-line entry of its name
- System call by another program
- Batch execution

When execution starts:

- OS loads the program into RAM
- Allocates necessary resources
- Creates a Process Control Block (PCB)
- Assigns CPU for execution

◆ Multiple Processes from Same Program

One program can create multiple processes.

Example:

- Multiple users opening the same web browser.
- Opening multiple instances of MS Word.
- Running the same program by different users.

Each process:

- Has its own memory space
- Has its own registers and program counter
- Executes independently

◆ **Example**

If 5 users execute the same program:

- There is one program stored on disk
- But 5 separate processes are created in memory

S.N.	Component & Description
1	**Stack** The process Stack contains the temporary data such as method/function parameters, return address and local variables.
2	**Heap** This is dynamically allocated memory to a process during its run time.
3	**Text** This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.
4	**Data** This section contains the global and static variables.

States of a Process in Operating System

When a process is executed, it goes through different states during its lifetime. These states represent the current activity of the process.

1 New

- The process is being created.
- OS initializes the Process Control Block (PCB).

---

2 Ready

- The process is ready to execute.
- Waiting in the ready queue for CPU allocation.

---

3 Running

- The process is currently being executed by the CPU.
- Only one process per CPU can be in running state at a time.

---

4 Waiting (Blocked)

- The process is waiting for some event to occur.
- Example: Waiting for I/O operation, file access, or user input.

---

5 Terminated (Exit)

- The process has finished execution.
- OS releases allocated resources.

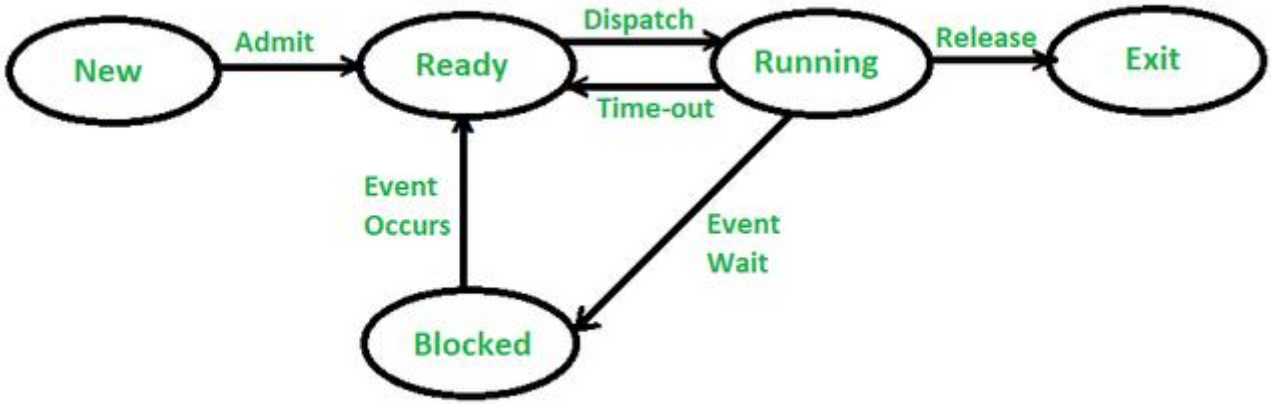
◆ State Transitions

- **New → Ready** : Process admitted to ready queue
- **Ready → Running** : CPU allocated by scheduler
- **Running → Waiting** : Waiting for I/O
- **Waiting → Ready** : I/O completed
- **Running → Terminated** : Process completes execution

Execution of Process in Five-state Model

This model consists of five states i.e, running, ready, blocked, new, and exit. The model works when any new job/process occurs in the queue, it is first admitted in the queue after that it goes in the ready state. Now in the Ready state, the process goes in the running state. In the running state, a process has two conditions i.e., either the process goes to the event wait or the process gets a time-out.

If the process has timed out, then the process again goes to the ready state as the process has not completed its execution. If a process has an event wait condition then the process goes to the blocked state and after that to the ready state. If both conditions are true, then the process goes to running state after dispatching after which the process gets released and at last it is terminated.



Possible State Transitions

There can be various events that lead to a state transition for a process. The possible state transitions are given below:

- **Null -> New:** A new process is created for the execution of a process.
- **New -> Ready:** The system will move the process from new to ready state and now it is ready for execution. Here a system may set a limit so that multiple processes can't occur otherwise there may be a performance issue.
- **Ready -> Running:** The OS now selects a process for a run and the system chooses only one process in a ready state for execution.
- **Running -> Exit:** The system terminates a process if the process indicates that is now completed or if it has been aborted.
- **Running -> Ready:** The reason for which this transition occurs is that when the running process has reached its maximum running time for uninterrupted execution. An example of this can be a process running in the background that performs some maintenance or other functions periodically.
- **Running -> Blocked:** A process is put in the blocked state if it requests for something it is waiting. Like, a process may request some resources that might not be available at the time or it may be waiting for an I/O operation or waiting for some other process to finish before the process can continue.

- **Blocked -> Ready:** A process moves from blocked state to the ready state when the event for which it has been waiting.
- **Ready -> Exit:** This transition can exist only in some cases because, in some systems, a parent may terminate a child's process at any time.

Advantages

- The New and Exit state are very useful constructs for managing the process.
- It is an efficient way of the previous two-state process model.

Disadvantages

- If a process gets terminated or exit from the OS, its data is not preserved by the OS.
- There can be a performance issue as in a situation where each process goes to a blocked state the CPU stays idle until at least one process leaves the waiting state.

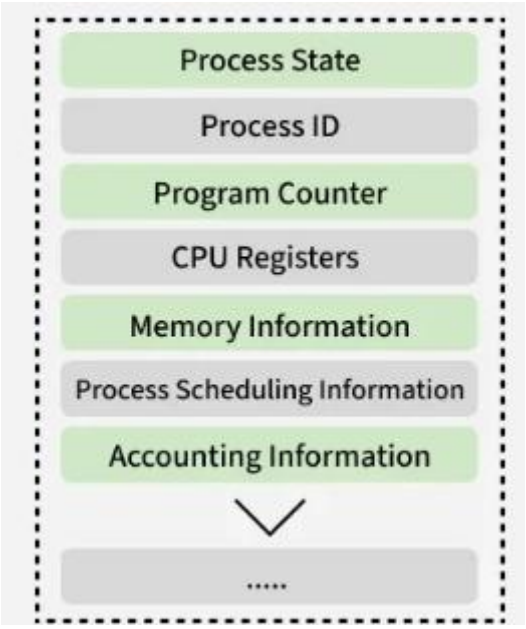
Process Control Block in OS

A Process Control Block (PCB) is a data structure used by the operating system to keep track of process information and manage execution. It helps the OS monitor and control process execution

- Each process is given a unique Process ID (PID) for identification.
- The PCB stores details such as process state, program counter, stack pointer, open files, and scheduling info.
- During a state transition, the OS updates the PCB with the latest execution data.
- It also includes register values, CPU quantum, and process priority.
- The Process Table is an array of PCBs that maintains information for all active processes.

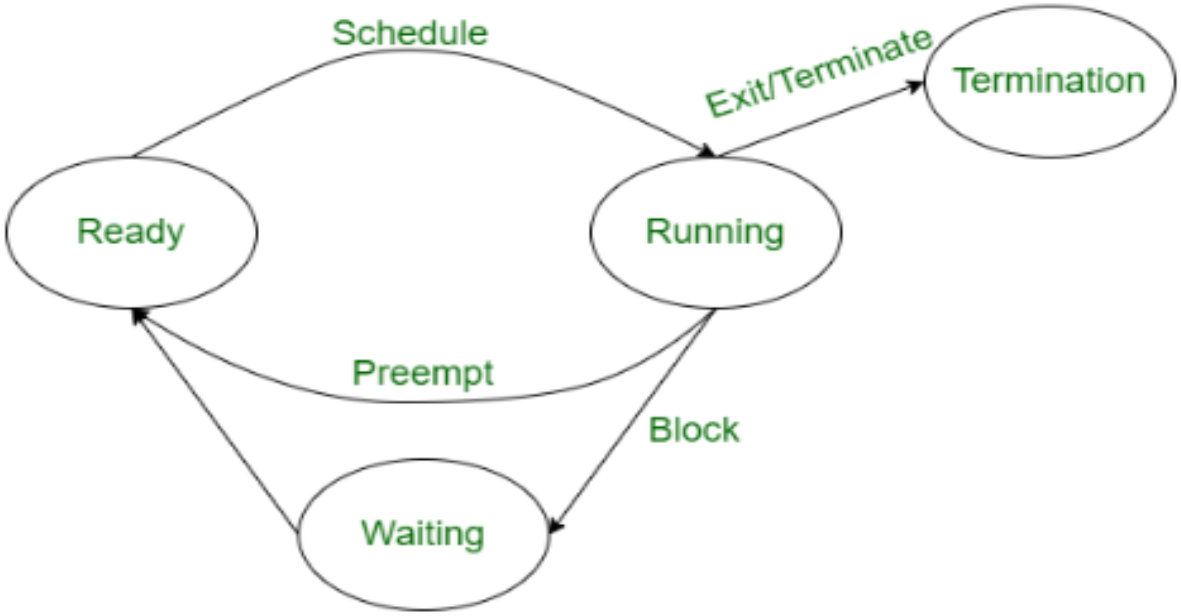
Terminologies used in Process Control Block

- **Process state:** Stores whether the process is running, waiting, ready, or terminated
- **Process number Or PID:** Every process is assigned a unique id known as process ID or PID.
- **Program counter:** Program Counter stores the address of the next instruction that is to be executed for the process.
- **Register:** Registers in the PCB, it is a data structure. When a processes is running and it's time slice expires, the current value of process specific registers would be stored in the PCB and the process would be swapped out. When the process is scheduled to be run, the register values is read from the PCB and written to the CPU registers. This is the main purpose of the registers in the PCB.
- **Memory limits:** This field contains the information about memory management system used by the operating system. This may include page tables, segment tables, etc.
- **List of Open files:** This information includes the list of files opened for a process.



Operations on a Process

In an Operating System, various operations can be performed on a process during its lifecycle. These operations are managed by the OS to ensure proper execution and resource utilization.



◆ 1 Process Creation

- A process is created when a program is executed.

- It can be created by:
  - System initialization
  - User request
  - Batch job
  - Another process (parent process)

Steps involved:

- Assign Process ID (PID)
- Allocate memory
- Initialize Process Control Block (PCB)
- Add process to ready queue

☞ Example: A user opening a browser application.

◆ 2 Process Execution

- The OS scheduler selects a process from the ready queue.
- CPU is allocated for execution.
- Process moves from **Ready → Running** state.

◆ 3 Process Waiting (Blocking)

- If a process needs I/O or some resource, it moves to waiting state.
- After completion of I/O, it returns to ready state.

◆ 4 Process Termination

A process terminates when:

- It completes execution normally.
- It encounters an error.
- It is terminated by another process or OS.
- Parent process terminates it.

After termination:

- Memory and resources are released.
- PCB is deleted.

◆ 5 Process Suspension and Resumption

- OS may temporarily suspend a process to manage memory.
- Suspended process is stored on disk.
- It can later be resumed and moved back to ready state.

What is a Thread?

A **Thread** is the smallest unit of execution within a process. It represents a separate path of execution inside a program.

A thread is also called a **lightweight process (LWP)** because it requires fewer resources than a full process.

Definition

A thread is an independent sequence of instructions that can be scheduled and executed concurrently by the operating system.

◆ Key Characteristics of a Thread

- Exists within a process
- Shares memory and resources of the parent process
- Has its own:
  - Program Counter
  - Register set
  - Stack
- Shares:
  - Code section
  - Data section
  - Heap memory
  - Open files

◆ Important Points

- The operating system creates and manages threads.
- Multiple threads within the same process can execute concurrently.
- Threads improve performance by enabling parallelism.
- Each thread belongs to exactly one process.
- A thread cannot exist independently outside a process.

◆ Advantages of Threads

- Faster context switching than processes
- Efficient resource sharing
- Better CPU utilization
- Improves application responsiveness

Difference Between Process and Thread

The main difference between a **process** and a **thread** lies in memory usage and resource sharing.

◆ Memory Space

- **Processes** run in separate memory spaces.
- **Threads** within the same process run in a shared memory space.

☞ This means processes are independent, while threads are interdependent.

◆ Resource Sharing

Threads share the following with other threads of the same process:

- Code section
- Data section
- Heap memory
- Open files
- Signals
- OS resources

Processes do not share these resources by default.

◆ Independent Components

Even though threads share resources, each thread has its own:

- Program Counter (PC)
- Register Set

- Stack Space

This allows multiple threads to execute different parts of a program simultaneously.

Similarly, each process also has its own:

- Memory space
- Registers
- Stack
- Resources

◆ **Comparison Table**

Feature	Process	Thread
Memory Space	Separate	Shared (within same process)
Resource Sharing	No sharing	Shares code, data, resources
Creation Cost	High	Low
Context Switching	Slower	Faster
Communication	Inter-Process Communication (IPC) required	Direct communication

Threads and its Types in Operating System

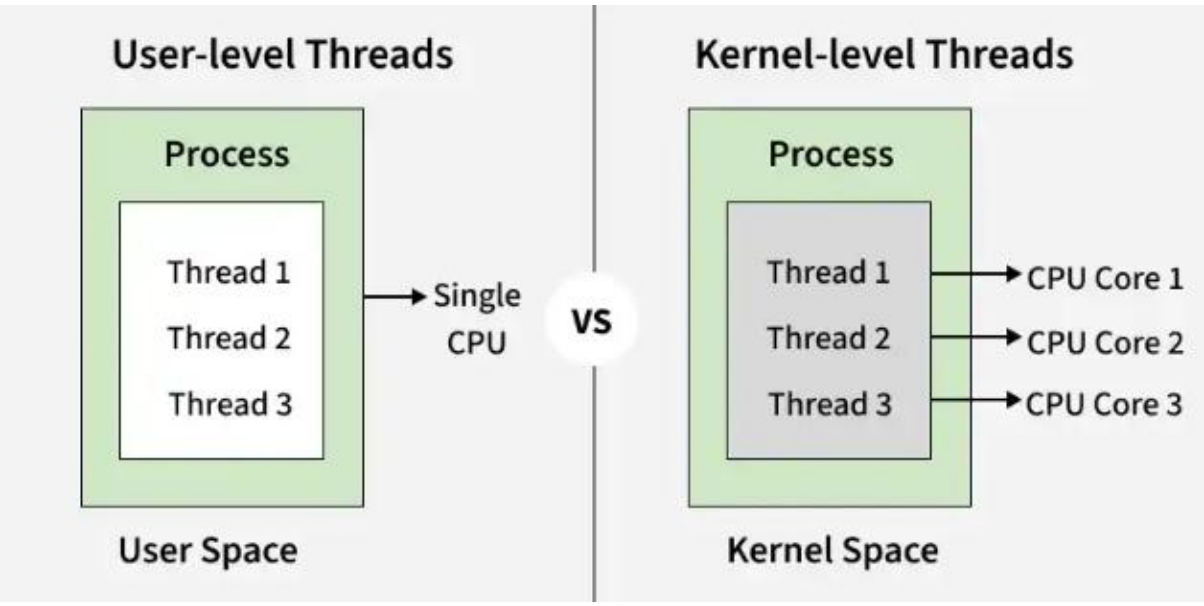
A thread is a single sequence stream within a process and is called a lightweight process because it is smaller and faster. It allows multiple tasks to run simultaneously, improving program efficiency.

- On single-core systems it creates an illusion of parallelism; on multi-core systems it executes in true parallel.
- Each thread has its own program counter, register set, and stack space.
- Threads share code, data, and operating system resources within the same process.

Types of Threads

Threads are mainly classified based on how they are managed and scheduled in an operating system. There are two primary types of threads.

1. User-Level Threads (ULT)
2. Kernel-Level Threads (KLT)



User Level Thread (ULT)

User-Level Threads (ULTs) are managed by a thread library in user space without direct involvement of the operating system kernel. Since user-level threads rely on the thread library, any operation that stalls one thread, such as I/O or a page fault, can suspend all threads within the process and they cannot fully utilize multiple processors.

- Managed in user space by a thread library.
- Faster creation and context switching.
- Cannot fully exploit multiprocessor parallelism.

Advantages	Disadvantages
Can be implemented on operating systems that do not support multithreading.	No or very limited coordination between threads and the kernel.

Advantages	Disadvantages
Simple representation with only program counter, register set, and stack space.	If one thread causes a page fault, the entire process blocks.
Easy to create since no kernel intervention is required.	A blocking system call blocks all threads within the process.
Fast thread switching because no operating system calls are needed.	Cannot fully utilize multiprocessor systems for parallel execution.

Kernel Level Thread (KLT)

Kernel-Level Threads (KLTs) are threads that are directly controlled and managed by the operating system kernel. They support real parallelism on multi-core processors since the OS scheduler handles their execution. The kernel is responsible for thread creation, scheduling, and synchronization. Although they provide better CPU utilization and prevent one blocked thread from affecting others, they involve higher overhead and slower context switching compared to User-Level Threads (ULTs).

- Completely managed and scheduled by the operating system kernel.
- A blocked thread does not stop other threads in the same process.
- Context switching is slower due to kernel mode involvement.

Advantages of KLT	Disadvantages of KLT
The kernel has complete information about all threads, allowing better scheduling decisions.	Slower and less efficient due to kernel involvement.
Suitable for applications that frequently perform blocking operations.	Context switching requires kernel mode, increasing overhead.
If one thread blocks, other threads in the same process can continue execution.	Requires a Thread Control Block (TCB), adding memory overhead.

Advantages of KLT	Disadvantages of KLT
Can utilize multiple CPU cores for true parallel execution.	Creation and management are more time-consuming compared to ULTs.

Threading Issues

- **fork() and exec()**: In multithreaded programs, fork() may duplicate all threads or just the calling thread, depending on the system. exec() replaces the entire process—including all threads—with the new program.
- **Signal Handling**: Signals notify a process of events. They can be synchronous or asynchronous and are handled by either the default kernel handler or a user-defined handler.
- **Thread Cancellation**: Threads can be terminated before completion. Cancellation can be **asynchronous** (immediate) or **deferred** (thread checks periodically). Example: stopping all threads loading a webpage.
- **Thread-Local Storage (TLS)**: Threads share process data, but sometimes need private copies, such as unique identifiers for transactions.
- **Scheduler Activations**: The kernel provides virtual processors, allowing a user-thread library to schedule threads efficiently.

Advantages of Threading

- **Responsiveness:** A multithreaded application increases responsiveness to the user.
- **Resource Sharing:** Resources like code and data are shared between threads, thus allowing a multithreaded application to have several threads of activity within the same address space.
- **Increased Concurrency:** Threads may be running parallelly on different processors, increasing concurrency in a multiprocessor machine.
- **Lesser Cost:** It costs less to create and context-switch threads than processes.
- **Lesser Context-Switch Time:** Threads take lesser context-switch time than processes.

Disadvantages of Threading

- **Complexity:** Threading can make programs more complicated to write and debug because threads need to synchronize their actions to avoid conflicts.
- **Resource Overhead:** Each thread consumes memory and processing power, so having too many threads can slow down a program and use up system resources.
- **Difficulty in Optimization:** It can be challenging to optimize threaded programs for different hardware configurations, as thread performance can vary based on the number of cores and other factors.
- **Debugging Challenges:** Identifying and fixing issues in threaded programs can be more difficult compared to single-threaded programs, making troubleshooting complex.

Multithreading in Operating System

◆ Introduction

Multithreading is a technique in which a single process is divided into multiple threads to perform multiple tasks concurrently. It allows an application to execute several parts of a task simultaneously.

◆ Definition

Multithreading enables a process to contain more than one thread of execution. Each thread performs a separate task but shares the same memory and resources of the parent process.

◆ Need for Multithreading

In a **single-threaded system**, only one task can be performed at a time. This leads to:

- Poor CPU utilization
- Slow response time
- Idle waiting during I/O operations

To overcome these limitations, **multithreading** is introduced.

◆ How Multithreading Works

- A process is divided into multiple threads.
- Each thread executes independently.
- Threads share:
  - Code section
  - Data section
  - Files and resources
- Each thread has its own:
  - Program Counter
  - Register set
  - Stack

◆ **Advantages of Multithreading**

1 Improved responsiveness
2 Better CPU utilization
3 Faster execution
4 Efficient resource sharing
5 Supports multitasking

◆ **Example**

In a web browser:

- One thread handles user interaction
- One thread loads web pages
- One thread downloads files

All threads run concurrently within the same process.

Multithreading Models (Thread Mapping Models)

Thread models describe the relationship between **user-level threads** and **kernel-level threads**. The three main models are:

1 Many-to-One Thread Model

◆ **Introduction**

The **Many-to-One model** maps multiple user-level threads to a single kernel-level thread. In this model, thread management is handled entirely in user space, and the operating system is not aware of multiple threads.

◆ **Working**

- Many user threads → mapped to one kernel thread
- Thread scheduling is performed by a user-level thread library
- The kernel schedules only one thread (the entire process)

◆ **Features**

- Thread management done in user space
- Fast context switching
- Can be implemented even on simple kernels without thread support

- Only one kernel-level thread exists

◆ Advantages

1 Efficient context switching
2 Low overhead
3 Simple to implement
4 Does not require kernel support for threads

◆ Disadvantages

1 No true parallelism on multi-core systems
2 Cannot utilize hardware acceleration of multiprocessor systems
3 If one thread performs a blocking system call, the entire process blocks
4 Entire system may stop if blocking occurs

2 Many-to-One Thread Model

◆ Introduction

The **Many-to-One model** maps multiple user-level threads to a single kernel-level thread. In this model, thread management is handled entirely in user space, and the operating system is not aware of multiple threads.

◆ Working

- Many user threads → mapped to one kernel thread
- Thread scheduling is performed by a user-level thread library
- The kernel schedules only one thread (the entire process)

◆ Features

- Thread management done in user space
- Fast context switching
- Can be implemented even on simple kernels without thread support
- Only one kernel-level thread exists

◆ **Advantages**

1 Efficient context switching
2 Low overhead
3 Simple to implement
4 Does not require kernel support for threads

◆ **Disadvantages**

1 No true parallelism on multi-core systems
2 Cannot utilize hardware acceleration of multiprocessor systems
3 If one thread performs a blocking system call, the entire process blocks
4 Entire system may stop if blocking occurs

3 Many-to-Many Multithreading Model

◆ **Introduction**

The **Many-to-Many model** maps multiple user-level threads to multiple kernel-level threads. The number of kernel threads depends on the specific application and system requirements.

This model is considered a **compromise between Many-to-One and One-to-One models**.

◆ **Working**

- Many user threads ↔ Many kernel threads
- Thread management is shared between user-level library and kernel
- The number of kernel threads may be equal to or less than user threads

---

◆ **Features**

- Developer can create multiple user and kernel threads
- Flexible thread mapping
- Supports better concurrency than Many-to-One model
- Kernel can schedule another thread if one thread blocks

◆ **Advantages**

1 Better performance and flexibility
2 If one thread makes a blocking system call, other threads can continue
3 Efficient utilization of system resources
4 Reduced overhead compared to One-to-One model

◆ Disadvantages

1 Complex to implement
2 Thread management is more complicated
3 True concurrency may still be limited depending on kernel scheduling

Concurrency in Operating System

◆ Introduction

Concurrency is the execution of multiple instruction sequences at the same time. It occurs in an operating system when several processes or threads are running simultaneously.

Concurrency improves system performance and resource utilization by allowing multiple tasks to progress during overlapping time periods.

◆ How Concurrency Occurs

- Multiple processes exist in the system.
- CPU switches rapidly between processes (context switching).
- In multi-core systems, processes may truly run in parallel.
- Threads within the same process may execute concurrently.

◆ Communication Between Processes

Concurrent processes or threads communicate using:

1 **Shared Memory**

- Processes share common memory space.
- Requires synchronization mechanisms.

2 **Message Passing**

- Processes exchange messages.
- No shared address space required.

Problems in Concurrency

1 Non-Atomic Operations

Non-atomic operations are those that can be interrupted during execution.
If multiple processes access shared data and an operation is interrupted, it may lead to inconsistent or incorrect results.

☞ Example: Updating a shared variable without synchronization.

---

2 Race Condition

A race condition occurs when the final outcome depends on which process executes first.
If two or more processes access shared data simultaneously without proper control, unexpected results may occur.

☞ The result "races" between processes.

---

3 Blocking

Blocking occurs when a process waits for a resource or event (like I/O operation).
The process cannot continue execution until the resource becomes available.
Long blocking may reduce system performance.

---

4 Starvation

Starvation happens when a process never gets the required resources to proceed.
It is usually caused by improper scheduling policies where high-priority processes continuously get CPU time.

---

5 Deadlock

Deadlock occurs when two or more processes are waiting for each other's resources and none can proceed.
In this situation, all involved processes remain blocked permanently.

Advantages of Concurrency

1 Improved CPU Utilization

While one process waits for I/O, another process can use the CPU.

2 Increased Throughput

More tasks can be completed in less time.

3 Better Responsiveness

Interactive applications respond faster to user input.

4 Efficient Resource Sharing

Multiple processes can share memory, files, and devices efficiently.

5 Supports Multitasking

Allows multiple applications to run at the same time.

6 Reduced Idle Time

Minimizes CPU idle time during I/O operations.

Disadvantages of Concurrency

1 Race Conditions

Incorrect results may occur if shared data is not properly synchronized.

2 Deadlock

Processes may wait indefinitely for each other's resources.

3 Starvation

Some processes may never get CPU or required resources.

4 Increased Complexity

Program design and debugging become more difficult.

5 Synchronization Overhead

Extra mechanisms like semaphores and locks are required.

6 Resource Contention

Multiple processes competing for limited resources may reduce performance.

Types of Processes in Operating System

A process in an operating system can be classified into two types:

1 **Independent Process**
2 **Co-operating Process**

---

1. Independent Process

- An independent process does **not share data** with other processes.
- It **cannot affect** and is **not affected** by other executing processes.
- It executes in its own address space.
- No need for Inter-Process Communication (IPC).

✅ Example:

A calculator application running separately from a text editor.

✓ Advantages:

- Simple design
- No synchronization issues
- No race conditions

---

2. Co-operating Process

- A co-operating process **can affect or be affected** by other processes.
- It shares data, resources, or information.

- Requires synchronization mechanisms.
- Uses **Inter-Process Communication (IPC)**.

✅ Example:

Client-server systems, producer-consumer problem.

Why Use Co-operating Processes?

Even though independent processes seem efficient, co-operating processes are useful because they:

✓ 1. Increase Computational Speed

Tasks can be divided into subtasks and executed in parallel.

✓ 2. Provide Modularity

Large systems can be divided into smaller cooperating modules.

✓ 3. Improve Convenience

Multiple users or programs can share data.

✓ 4. Enable Information Sharing

Processes can share common data like files or databases.

Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is a mechanism that allows processes to:

- Communicate with each other
- Share data
- Synchronize their execution

🔷 Two Main IPC Methods:

1. **Shared Memory**
2. **Message Passing**

Critical Section Problem in Operating System

What is a Critical Section?

A **critical section** is a part of a program where **shared resources** (like variables, files, memory, etc.) are accessed.

Since multiple processes may try to access shared data at the same time, it can lead to:

- Data inconsistency
- Race conditions
- Unexpected results

Therefore, only **one process should execute the critical section at a time**.

Structure of a Process (with Critical Section)

A process containing a critical section is divided into four parts:

1. **Entry Section** – Code where the process requests permission to enter the critical section.
2. **Critical Section** – Code where shared resources are accessed.
3. **Exit Section** – Code that releases the critical section.
4. **Remainder Section** – Rest of the code (non-critical work).

Critical Section Problem

The **critical section problem** is the problem of designing a protocol that ensures cooperative processes can access shared resources safely **without data inconsistency**.

Requirements of a Correct Solution

Any solution to the critical section problem must satisfy the following three conditions:

---

1 Mutual Exclusion

- If one process is executing in its critical section,
- **No other process** should be allowed to enter its critical section.

✓ Ensures data consistency
✓ Prevents race conditions

---

2 Progress

- If no process is in the critical section,
- And some processes want to enter,
- The selection of the next process **cannot be postponed indefinitely**.

✓ Avoids unnecessary waiting
✓ Ensures fair decision making

---

3 Bounded Waiting

- After a process requests entry into the critical section,
- There must be a limit on the number of times other processes can enter before it gets its turn.

✓ Prevents starvation
✓ Guarantees fairness

Example

Suppose two processes P1 and P2 want to update the same bank account balance:

Without proper synchronization:

- Both may read the same value.
- Both update independently.
- Final balance becomes incorrect.

With proper critical section implementation:

- Only one process updates at a time.
- Data remains consistent.

Semaphores in Process Synchronization

A semaphore is a synchronization tool used in operating systems to manage access to shared resources in a multi-process or multi-threaded environment. It is an integer variable that controls process execution using atomic operations like wait() and signal(). Semaphores help prevent race conditions and ensure proper coordination between processes.

- Controls entry into the critical section.
- Maintains a counter representing available resources.
- Ensures mutual exclusion among processes.
- Can block and wake up processes during execution.
- Widely used in concurrent programming.

How Semaphores Work
A semaphore in OS uses two primary atomic operations:

1. wait(S)

```
wait(S):
    S = S - 1          // take resource
    if S < 0:          // none available
        wait           // go to sleep
```

- Increments the semaphore value.
- If there are waiting processes, one is awakened.
- Used to release a resource.

Example: Let's consider two processes P1 and P2 sharing a semaphore S, initialized to 1:

Features of Semaphores

- **Mutual Exclusion**: Semaphore ensures that only one process accesses a shared resource at a time.
- **Process Synchronization**: Semaphore coordinates the execution order of multiple processes.
- **Resource Management**: Limits access to a finite set of resources, like printers, devices, etc.
- **Reader-Writer Problem**: Allows multiple readers but restricts the writers until no reader is present.

- **Avoiding Deadlocks**: Prevents deadlocks by controlling the order of allocation of resources.

Types of Semaphores
Semaphores are mainly of two Types:

1. Counting Semaphore

A counting semaphore can have values ranging from 0 to any positive integer. It is used when multiple instances of a resource are available and need to be managed.

- Value ranges from 0 to n.
- Manages multiple resource instances.
- Controls access to limited resources.
- Example: Managing access to 5 printers or 3 database connections.

2. Binary Semaphore

A binary semaphore has only two possible values: 0 and 1. It is mainly used for mutual exclusion, ensuring that only one process enters the critical section at a time.

- Value is either 0 or 1.
- Used for mutual exclusion.
- Similar to a mutex lock.
- Managing access to a single critical section

Limitations of Semaphores
- Priority Inversion: A low-priority process holding a semaphore can block a high-priority one.
- Deadlock: Processes may wait on each other's semaphores in a cycle, causing indefinite blocking.
- Complex to Manage: The OS must carefully track wait and signal calls; misuse can cause errors.
- Busy Waiting: In basic implementations, processes may keep checking the semaphore value, wasting CPU time.

Advantages and Disadvantages of Semaphores

◆ **Advantages of Semaphores**

1 Ensures Mutual Exclusion

Semaphores allow only **one process** to enter the critical section at a time (in case of binary semaphore).
This strictly follows the **mutual exclusion principle** and prevents race conditions.

---

2 No Busy Waiting (Efficient CPU Utilization)

In blocking semaphores:

- If a process cannot enter the critical section,
- It is moved to a **waiting queue** instead of continuously checking (spinning).

✓ Processor time is not wasted
✓ Better CPU efficiency compared to spinlocks

---

3 Machine Independent

Semaphores are implemented in the **operating system (microkernel)** and are generally machine independent.
They can be used across different hardware architectures.

---

4 Used for Synchronization & Resource Management

Semaphores are useful for:

- Solving **Producer–Consumer Problem**
- Handling **process synchronization**
- Managing limited resources

▼ Disadvantages of Semaphores

1 Complex Implementation

Semaphores are difficult to implement correctly.

- wait() and signal() must be executed in the correct order.
- Wrong implementation may lead to **deadlock**.

---

2 Loss of Modularity

The use of wait and signal operations:

- Spreads synchronization logic throughout the program.
- Makes the system less structured and harder to maintain.

This reduces modularity in large systems.

---

3 Possibility of Priority Inversion

A **low-priority process** may hold the semaphore while a **high-priority process** waits.

This situation is called **Priority Inversion**, and it affects system performance.

---

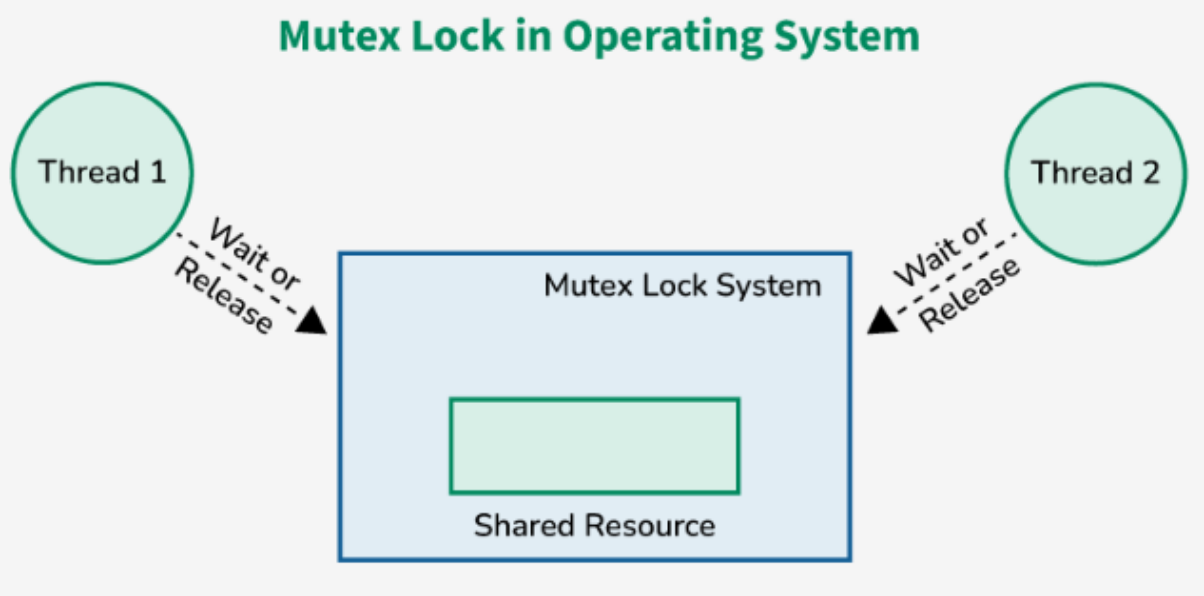
4 Deadlock & Starvation Risk

Improper use of semaphores can cause:

- **Deadlock**
- **Starvation**
- **Indefinite blocking**

Mutex

A mutex is different from a binary semaphore, which provides a locking mechanism. It stands for Mutual Exclusion Object. Mutex is mainly used to provide mutual exclusion to a specific portion of the code so that the process can execute and work with a particular section of the code at a particular time.



- A mutex enforces strict ownership. Only the thread that locks the mutex can unlock it.

- It is specifically used for locking a resource to ensure that only one thread accesses it at a time.
- Due to this strict ownership, a mutex is not only typically used for signaling between threads, but it is used for mutual exclusion also to ensuring that a resource is accessed by only one thread at a time.
- Mutex uses a priority inheritance mechanism to avoid priority inversion issues.

Advantages of Mutex

- No race condition arises, as only one process is in the critical section at a time.
- Data remains consistent and it helps in maintaining integrity.
- It is a simple locking mechanism that into a critical section and is released while leaving the critical section.

Disadvantages of Mutex

- If a thread holding the mutex is preempted or goes to sleep inside the critical section, other threads are blocked, which can lead to priority inversion or starvation.
- Mutexes involve context switching and kernel overhead, which makes them slower than spinlocks for very short critical sections.
- Incorrect usage (such as forgetting to unlock) can cause deadlock.

Monitors in Process Synchronization

Monitors are a high-level synchronization mechanism that simplify process and thread synchronization. They are built on top of locks and are mostly used in multithreading systems like Java.

Unlike semaphores, where the programmer must explicitly call **wait()** and **signal()**, monitors combine shared data and the operations on that data inside a single structure, making synchronization safer and easier to manage.

How to Implement Monitors

- Monitors are implemented at the programming language level, not directly by the operating system.
- In Java, monitor-like behavior is achieved using the synchronized keyword ensures that only one thread can execute inside the monitor at a time.
- A monitor encapsulates both shared data (critical resource) and the operations that access or modify it.
- Mutual exclusion is enforced automatically, unlike semaphores where programmers must explicitly call wait() and signal().
- Synchronization can be applied through synchronized methods or synchronized blocks.
- Condition variables are used to control thread waiting and signaling.
- The methods wait(), notify(), and notifyAll() provide process coordination.

```
class Monitor{                    //Name of Monitor

  variables;
  condition variables;

  process p1 {....}               //synchronized method
  process p2 {....}               //synchronized method

}
```

Readers-Writers Problem

The Readers-Writers Problem is a classic synchronization issue in operating systems. It deals with coordinating access to shared data (e.g., database, file) by multiple processes or threads.

- **Readers**: Multiple readers can read the shared data **simultaneously** without causing inconsistency (since they don't modify data).
- **Writers**: Only **one writer** can access the data at a time, and no readers are allowed while writing (to prevent data corruption).

The challenge is to design a synchronization scheme that ensures:

1. Multiple readers can access data together if no writer is writing.
2. Writers have exclusive access no other reader or writer can enter during writing.

Variants of the Problem

1. Readers Preference

- Readers are given priority.
- No reader waits if the resource is available for reading, even if a writer is waiting.
- Writers may suffer from starvation.

2. Writers Preference

- Writers are prioritized over readers.
- Ensures writers won't starve, but readers may wait longer.

Solution When Reader Has The Priority Over Writer

Here priority means, no reader should wait if the shared resource is currently open for reading. There are four types of cases that could happen here.

Case	Process 1	Process 2	Allowed/Not Allowed
Case 1	Writing	Writing	Not Allowed
Case 2	Writing	Reading	Not Allowed
Case 3	Reading	Writing	Not Allowed
Case 4	Reading	Reading	Allowed

Synchronization Variables

- **mutex:** Ensures mutual exclusion when updating the reader count (readcnt).
- **wrt:** Controls access to the shared data (used by both readers and writers).
- **readcnt:** Number of readers currently in the critical section.

Semaphore Operations

- **wait():** Decreases semaphore value, blocks if < 0.
- **signal():** Increases semaphore value, wakes waiting process if any.

Reader Process(Reader Preference)

1. Reader requests the entry to critical section.

2. If allowed:

- it increments the count of number of readers inside the critical section. If this reader is the first reader entering, it locks the **wrt** semaphore to restrict the entry of writers if any reader is inside.
- It then, signals mutex as any other reader is allowed to enter while others are already reading.
- After performing reading, it exits the critical section. When exiting, it checks if no more reader is inside, it signals the semaphore "wrt" as now, writer can enter the critical section.

3. If not allowed, it keeps on waiting.

```
do {
wait(mutex); // Lock before updating readcnt
readcnt++;
if (readcnt == 1)
wait(wrt); // First reader blocks writers
signal(mutex); // Allow other readers in
// ---- Critical Section (Reading) ----
wait(mutex);
readcnt--;
```

```
if (readcnt == 0)
signal(wrt); // Last reader allows writers
signal(mutex); // Unlock
} while(true);
```

Explanation:

- When a reader enters, it locks mutex to update readcnt.
- If it's the first reader, it locks wrt so writers are blocked.
- Multiple readers can now read the data simultaneously.
- When a reader exits, it decrements readcnt.
- If it's the last reader, it unlocks wrt so writers can proceed.

The first reader blocks writers, the last reader allows writers, and all readers in between share the resource. This gives preference to readers, but writers may starve.

Writer's Process

```
do {
wait(wrt); // Lock resource

// ---- Critical Section (Writing) ----

signal(wrt); // Release resource
} while(true);
```

Explanation:

- **wait(wrt):** Writer locks the resource to get exclusive access.
- **Critical Section:** Writer performs writing (no other reader or writer can enter).
- **signal(wrt):** Writer releases the resource after finishing.

Only one writer can write at a time, and no readers are allowed while writing. This ensures data integrity.

The Readers-Writers Problem highlights the need for proper synchronization when multiple processes access shared data.

- Readers Preference solution allows many readers to access simultaneously while blocking writers, improving read efficiency.
- However, writers may starve if readers keep arriving continuously.
- To avoid this, Writers Preference or a Fair solution (using queues or advanced semaphores) can be used to ensure no starvation and balanced access.

Producer Consumer Solution using Semaphores

The Producer-Consumer problem is a classic example of a synchronization problem in operating systems. It demonstrates how processes or threads can safely share resources without conflicts. This problem belongs to the process synchronization domain,

specifically dealing with coordination between multiple processes sharing a common buffer.

In this problem, we have:

- **Producers**: Generate data items and place them in a shared buffer.
- **Consumers**: Remove and process data items from the buffer.

The main challenge is to ensure:

1. A producer does not add data to a full buffer.
2. A consumer does not remove data from an empty buffer.
3. Multiple producers and consumers do not access the buffer simultaneously, preventing race conditions.

Semaphore: The Synchronization Tool

A semaphore is an integer-based signaling mechanism used to coordinate access to shared resources. It supports two atomic operations:

- **wait(S)**: Decreases the semaphore value by 1. If the value is ≤0, the process waits.
- **signal(S):** Increases the semaphore value by 1, potentially unblocking waiting processes.

```
wait(S){
while(S <= 0); // busy waiting
S--;
}
signal(S){
S++;
}
```

Problem Statement

Consider a fixed-size buffer shared between a producer and a consumer.

- The producer generates an item and places it in the buffer.
- The consumer removes an item from the buffer.

The buffer is the critical section. At any moment:

- A producer cannot place an item if the buffer is full.
- A consumer cannot remove an item if the buffer is empty.

To manage this, we use three semaphores:

- mutex – ensures mutual exclusion when accessing the buffer.
- full – counts the number of filled slots in the buffer.
- empty – counts the number of empty slots in the buffer.

Semaphore Initialization

```
mutex = 1; // binary semaphore for mutual exclusion
full = 0; // initially no filled slots
empty = n; // buffer size
```

Producer

```
do {
// Produce an item
wait(empty); // Check for empty slot
wait(mutex); // Enter critical section
// Place item in buffer
signal(mutex); // Exit critical section
signal(full); // Increase number of full slots
} while (true);
```

Consumer

```
do {
wait(full); // Check for filled slot
wait(mutex); // Enter critical section
// Remove item from buffer
signal(mutex); // Exit critical section
signal(empty); // Increase number of empty slots
} while (true);
```

Explanation:

- Empty ensures that producers don't overfill the buffer.
- Full ensures that consumers don't consume from an empty buffer.
- Mutex ensures mutual exclusion, so only one process accesses the buffer at a time.